Monday, June 15, 2026 8:17 PM

Bagging & Random FOREST

Agenda ⇒

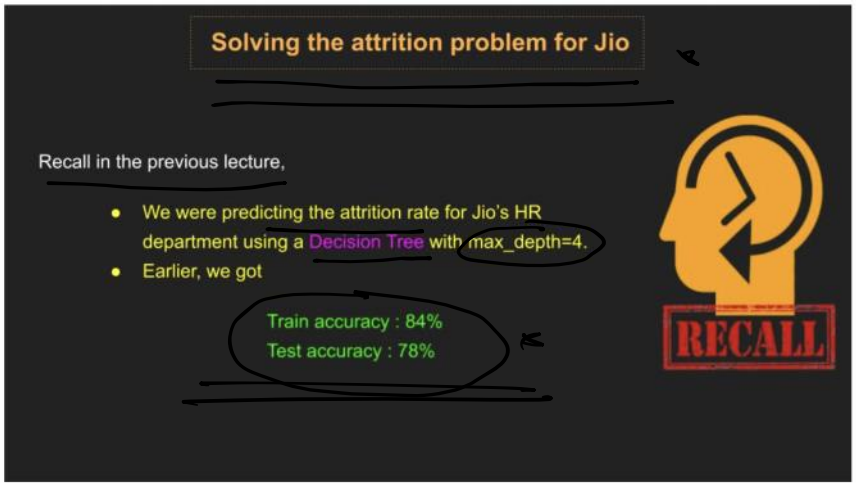
- Ensemble Learning
- Bagging Techniques
- Random Forest
- Hyperparam Tuning ⇒

max_depth → 1c

min_samples → split

PROBLEM Statement ⇒

Employee leaving or not



Reasons

Pruning

DTree → overfits very easily

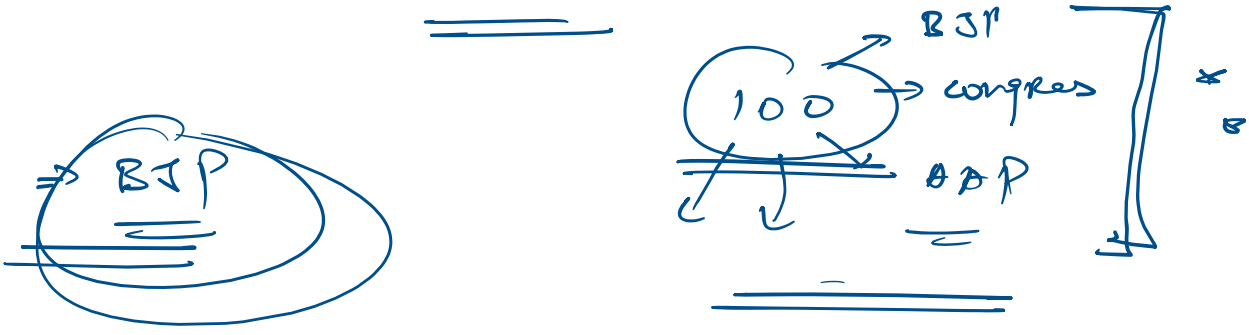
→ group of items

ensemble ⇒ multiple models + combine them smartly

(Base learners) → any independent model

DTRES

→ diff. as possible ⇒



⇒

⇒ multiple models will be combined so that we get a better model

pred^n → 20 models

1 model → pattern 1

2 model → pattern 2

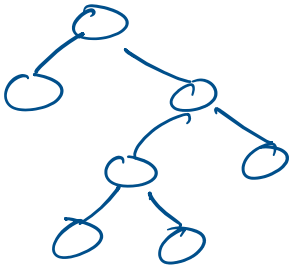
⋮

20 model → pattern 20

model final

different → Better

New → Pred



⇒

Ensemble Learning → Base learners + combine them

(1) Bagging ⇒

Types

(2) Boosting ⇒

3. Cascading
4. Stacking

* Bagging ⇒

Bootstrap → Aggregation ⇒ sum, mean, count, min

↓

sampling with replacement

* Bag → diff Balls (50)

* 1 Ball → note color

put that Back in Bag

Bagging to make a prediction

Regression → To predict Credit Score

⇒ 3 models ⇒

can be lin Reg, DTRes, kNN

(8) 1 Lin Reg 2 DTRes
1 DTRes 2 Lin Reg

M's

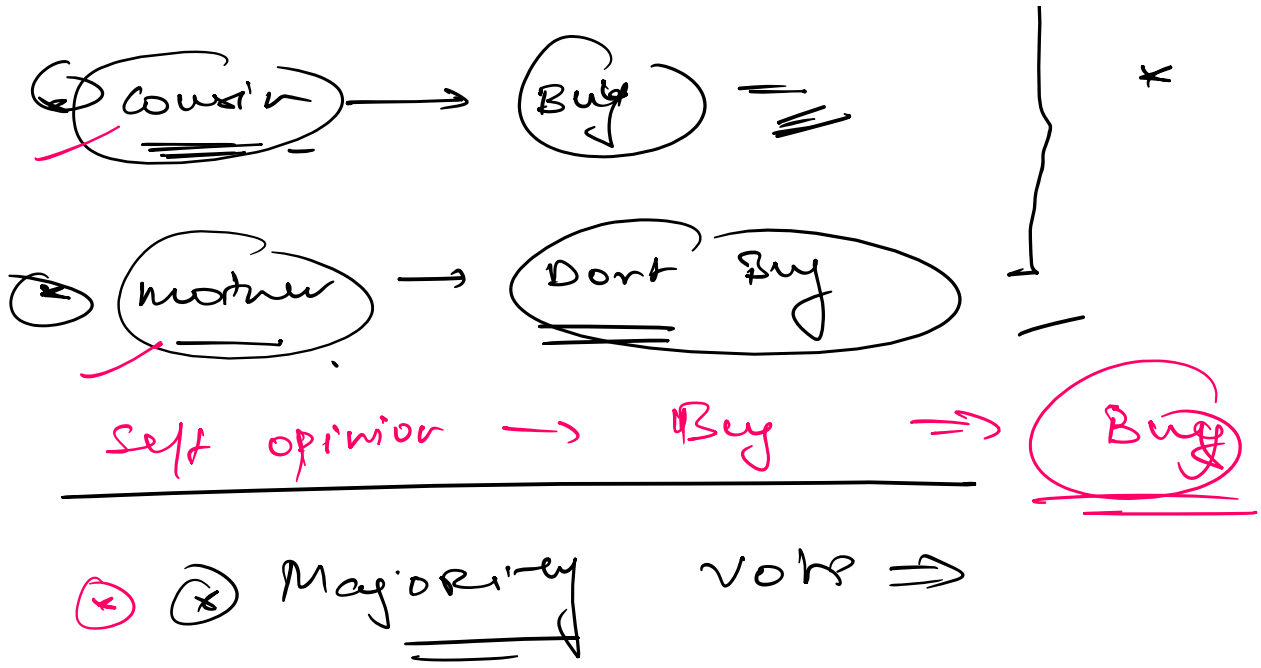
M1 → 82
M2 → 90
M3 → 78

avg | median

final pred = $\frac{82 + 90 + 78}{3}$ =

iphone → opinions

friend → Buy ⇒



diff opinions & diversity

Bagging

Bootstrap sampling

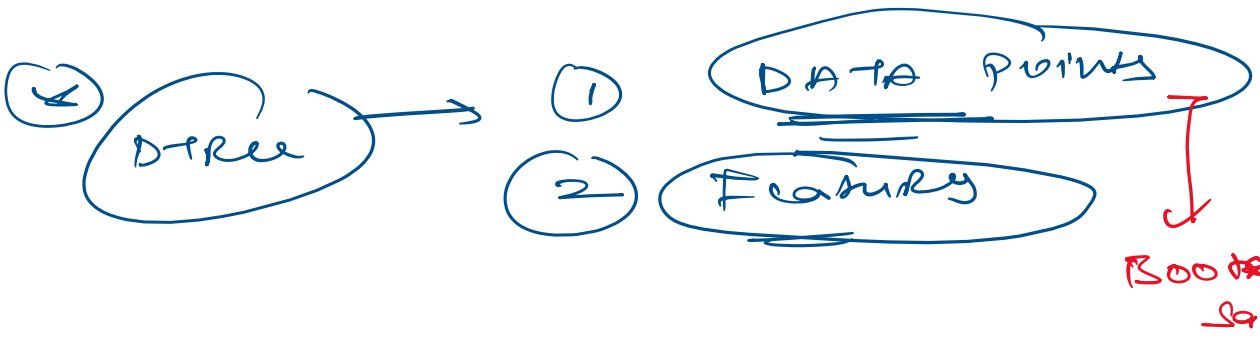
multiple Trees

RANDOM FORESTS ⇒ Type of Bagging

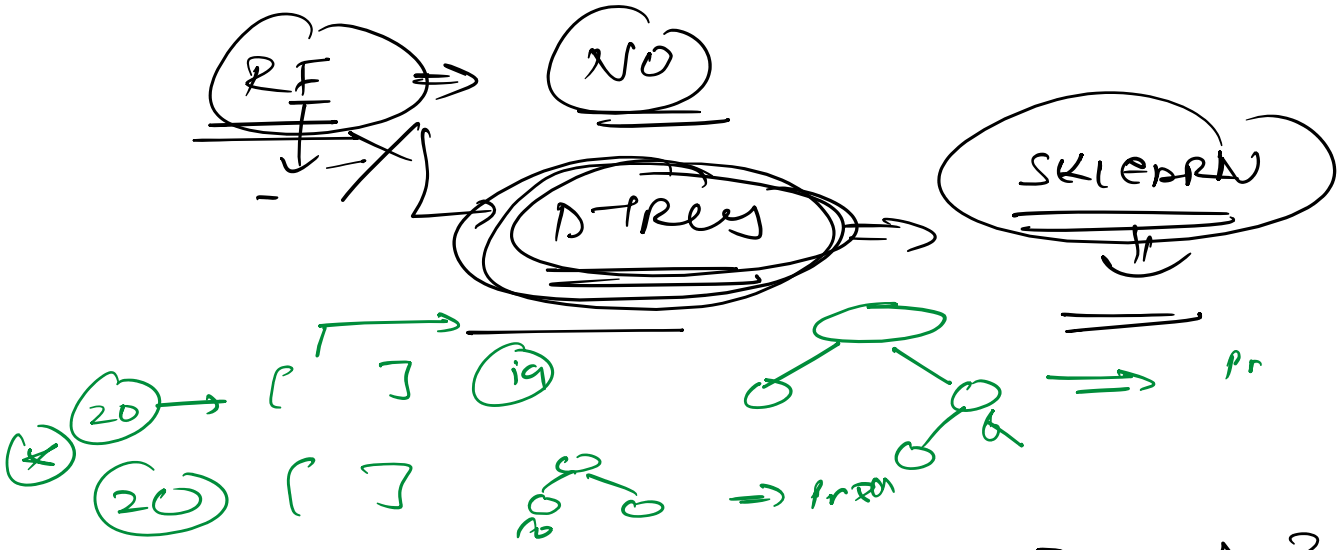
→ subset of DTrees

→

diff training data → every DTree

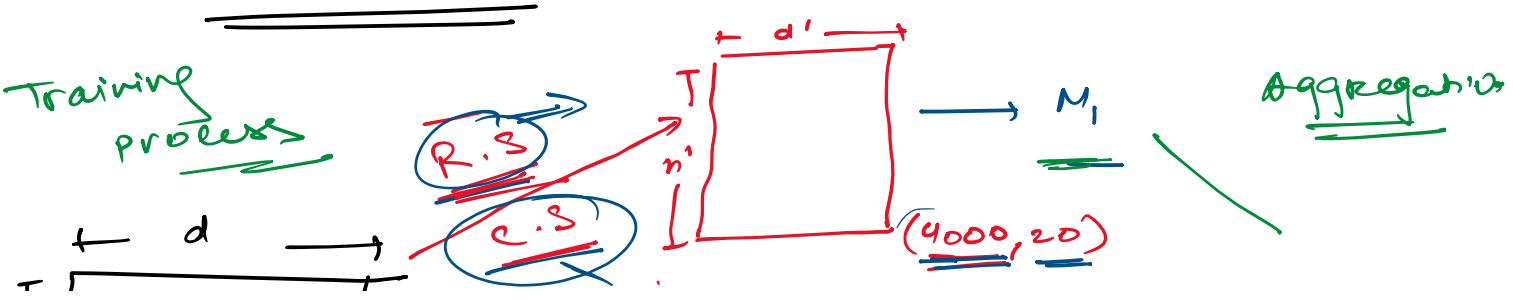


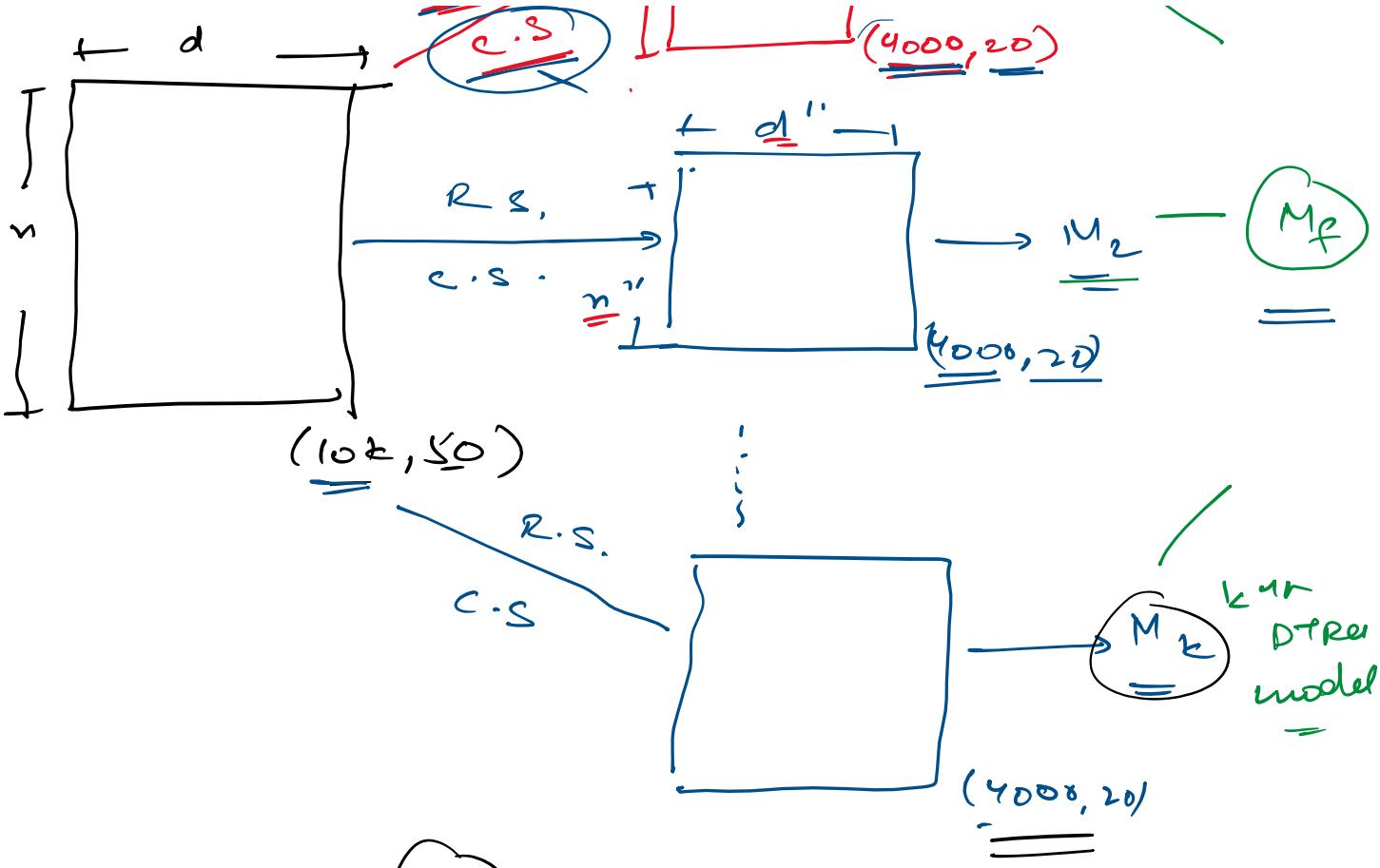
RF = Bunch of D-Trees + R.S. + C.S + Aggregation



How to Train Random Forest ?

→ employees leave or not





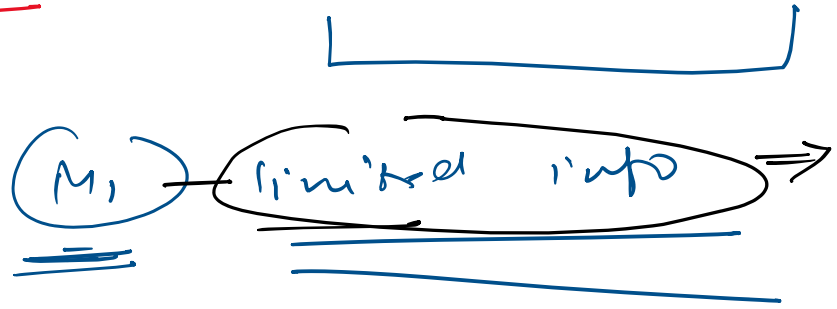
⇒ Bootstrap = True ⇒ Ra

DTree models → Similar i.e arch

* same depth, same structure

→ only data is different

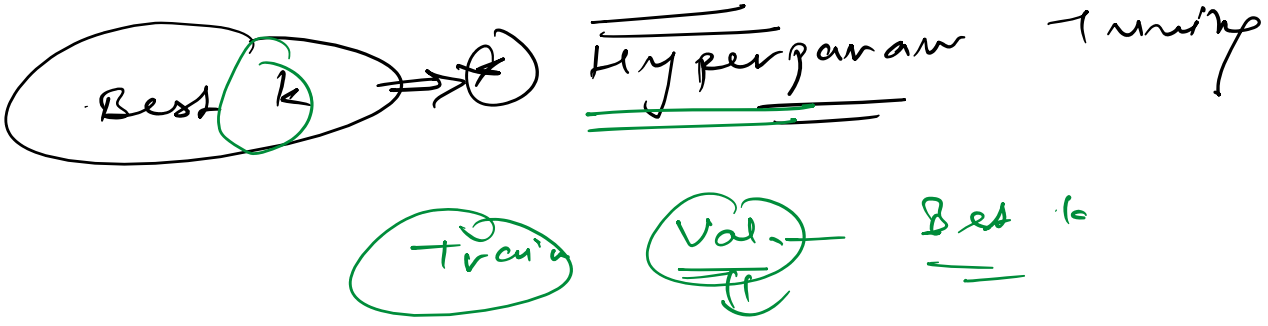
1st sampling → age, dist. from Home



2nd sampling → salary, years of exp

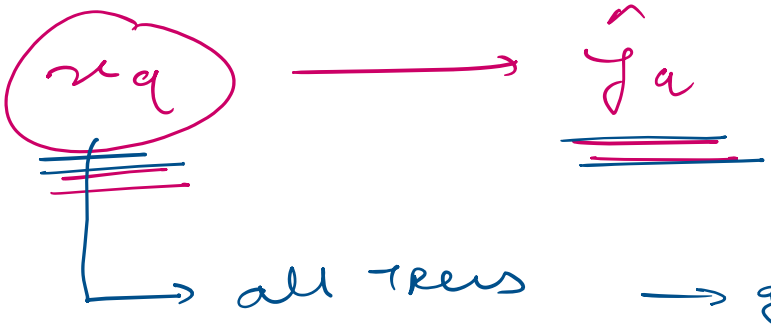


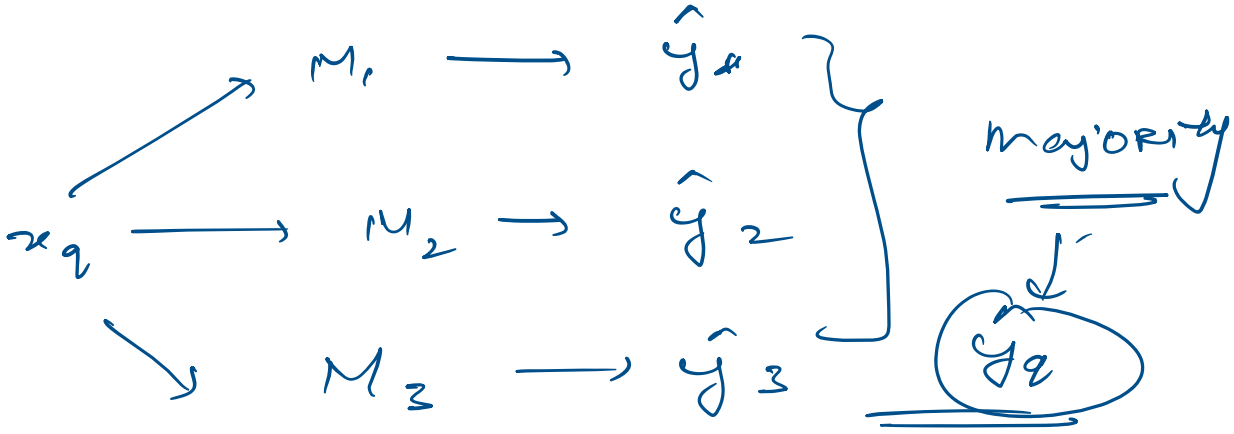
RF advantage → individual models diff. from each other



Inference ⇒







* accuracy ⇒

2 stages ⇒ training + inference

RF → 100 DTREs

RF → 100 DTRes

single model

(X) why do we need Randomness?

R.S. + C.S

(1) Reduce variance

(2) Reduce overfitting

(3) generalized model

individual DTRes → overfits very easily

overfitted DTRees

DTRee ⇒

↳ Deep → 0% Train error ←

Test very high

Bias ↓ , Variance ↑↑

DATA A → 50 lacs
B → 70 lacs
C → 40 lacs
] model unstable

Tree 1 → A B C D E → overfit
→ 0
2 → A C D F G
3 → B D E H i →

T_1 → 50
T_2 → 70
T_3 → 40
60

60
55

$$\frac{50 + 70 + 40 + 60 + 55}{5} = 55$$

Reduces variance

Columns — Age, BedRooms, Location, Area, dist. from metro

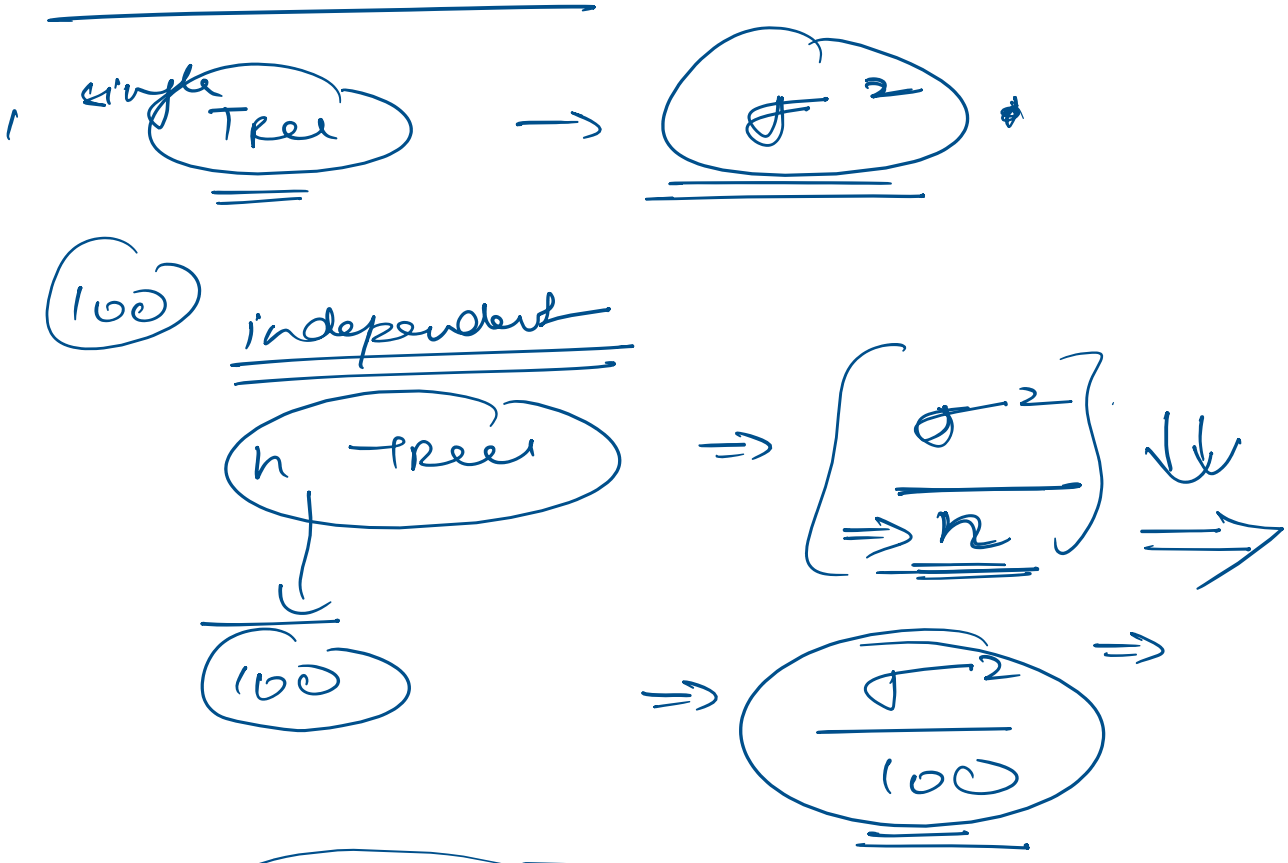
[ ] [ ] [ ] [ ]

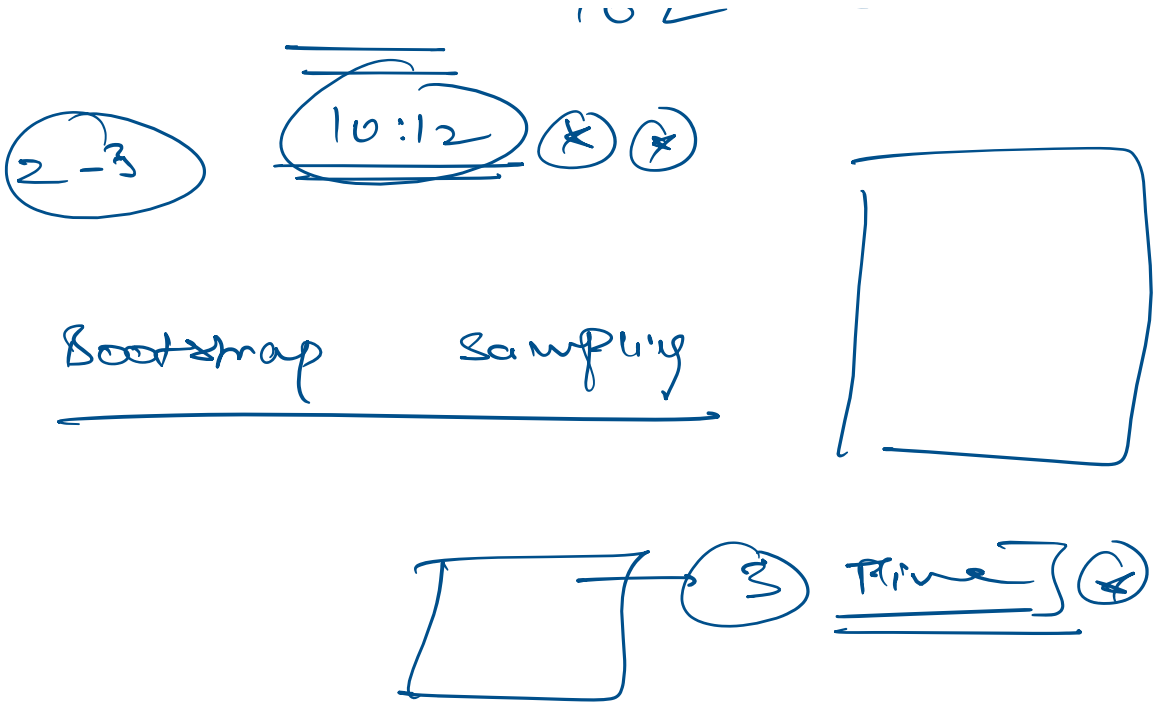
1 DTREE

[ ] Less correlated

[
Tree 1 → Area & Age
2 → Location BedRoom
3 →
]

Ro. → majority


single Tree
100
independent
n Tree
100
Tree
Tree
100
101
100
90
101



DTrees → fully grown Trees (large depth)

individual → overfitting

↓

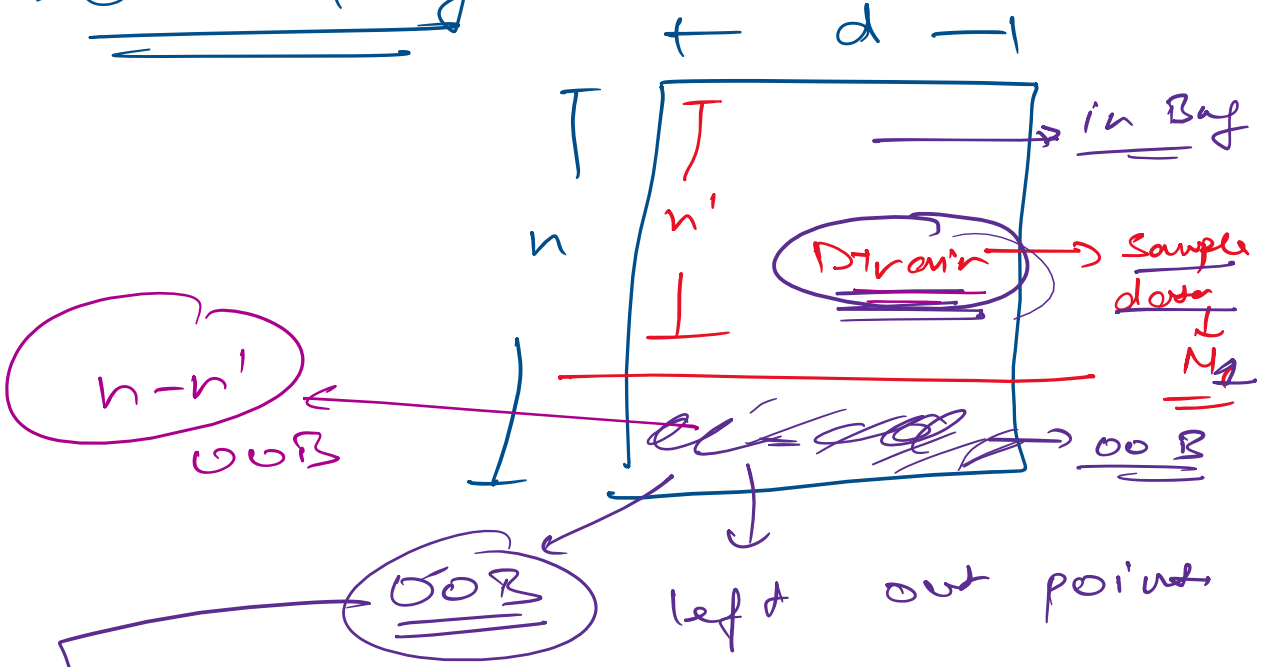
Reduce variance

↓

Randomness → R.S+C's

OOB Score

Out of Bag



out of Bag points = n-n'

validation

OOB → ✗ used in M_1

↳ validation data

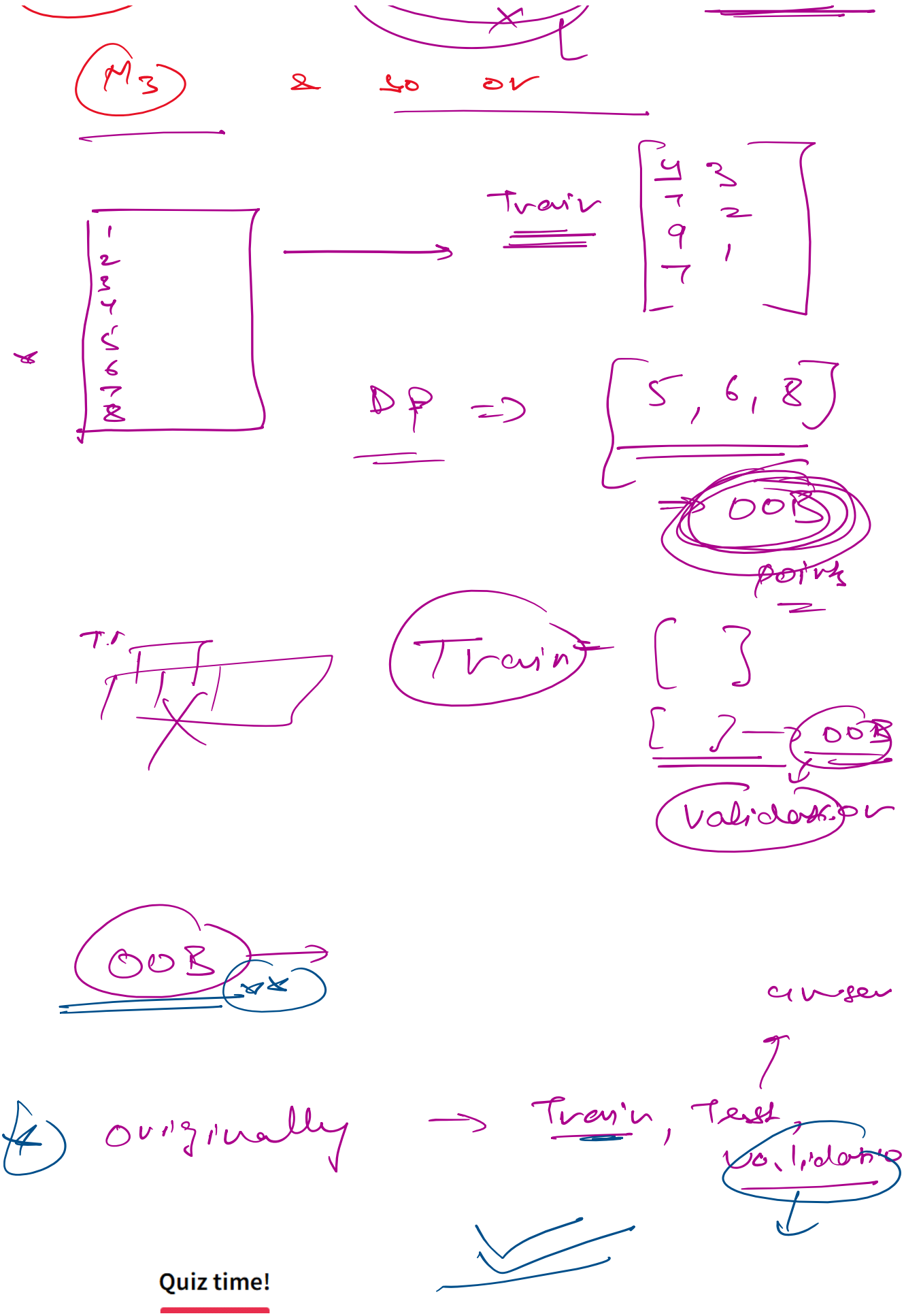
[ ]

M_1 → how good on OOB points

[ ]

M_2 → OOB → validation

✗


M3
Train
OOB
Validation
Quiz time!

Quiz time!

Time Left: 7s

If a dataset contains "n" rows, and "m" of these rows are sampled to train the base learners in Random Forest, what will be the cross-validation data for each of the models?

36 users have participated

A	Complete dataset with "n" rows	3%
B	A part of "m" sampled rows	19%
✓ C	Remaining "n-m" rows after sampling	78%
D	None of the above.	0%

End Quiz Now

m → Training

n − m

CV

Bias Variance Trade off ⇒

RF = DTree + R.S + C.S + Aggregation

little overfitted

Bias ↓

Variance ↓

⇒

Complex model

Randomness

Help me reduce variance

⇒ ✗ overfitted model → Bias ↓

=> (x) overfitted model → (risk) ↓

model is capable of making predictions

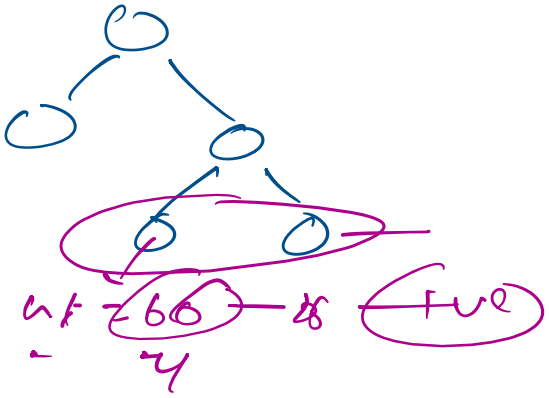
RF => Bias ↓ Variance ↓

Tree 1 →

Tree 2

100

$\sqrt{100} = 10$



Time Left: 11s

The target attributes in the random forest model indicates the value of?

24 users have participated

A	Leaf node	42%
B	Root node	13%
C	Decision node	37%
D	None of these	8%

End Quiz Now

max_samples => [0, 1]

max - samples

1 → 100 %

0.2 → 20 % of the data

DTrees → diff data

↓ max-samples ⇒

overfitting will reduce

c.s

max-features ↓ ⇒ ↓

[∄

* 1 tree

* 100 trees

more expensive

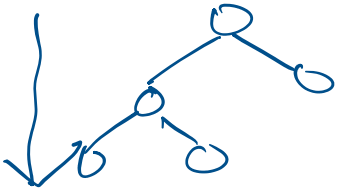
Train | Test complexity

1 DT → train time complexity ⇒ O(n log n × d)

numeric → sort → n log n

RF → k DTrees ⇒ O(k . n log n . d)

* Dist . compute ⇒



inference ⇒

* 1 DTree ⇒ O(max_depth)

RF → k trees ⇒ O(k . max depth)

* k = 50 / 100 / 200

not very expensive



RF can Be Trivially parallelized

M_1 M_2 M_k

Trained parallel

n_jobs = -1

Hyperparam Tuning ⇒

* max_depth = [- - - - -]

Best k

(*) ——————

Train, val-score

Best k

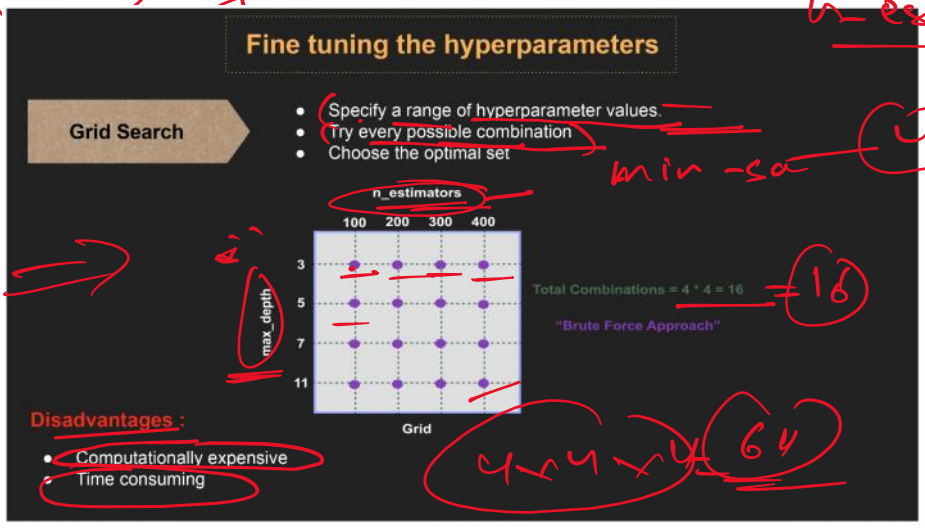
⇒ max_depth, n_estimators, ...

↳ find all of these at same time

(1) GRID SEARCH

max_depth = {4, 10, 11}

n_estimator = {100, 200, 400}



min-sa (4)

= 16

4×4×4 = 64

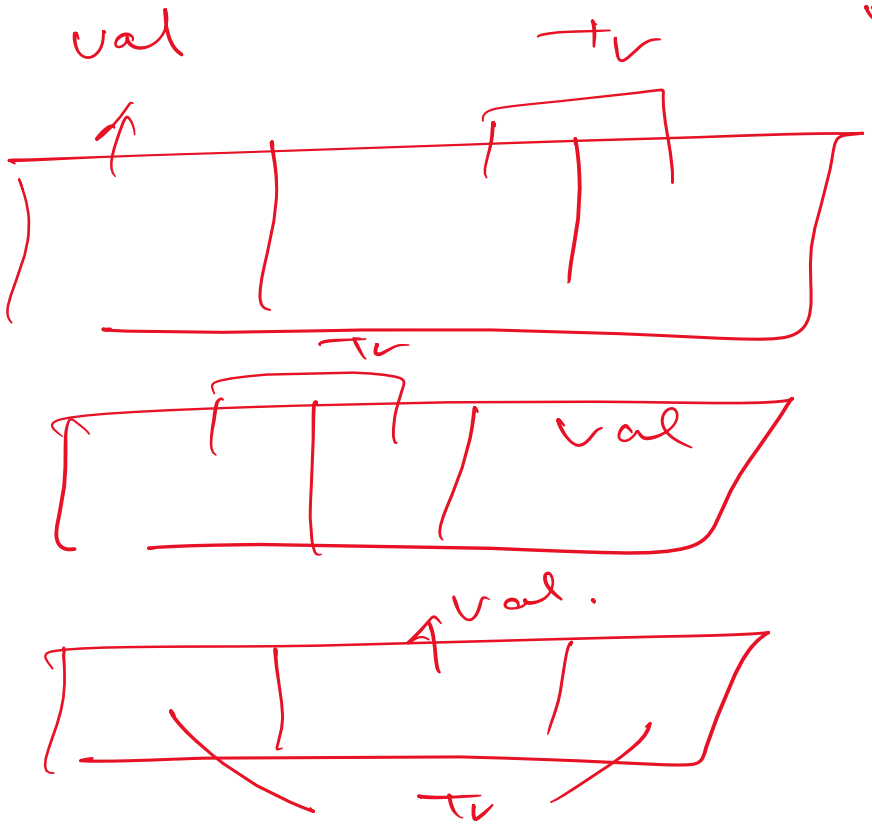
4, 100
4, 200
4, 400
10, 100
10, 200

3×3 = 9 values

16 → 16 models → scores

Best val score

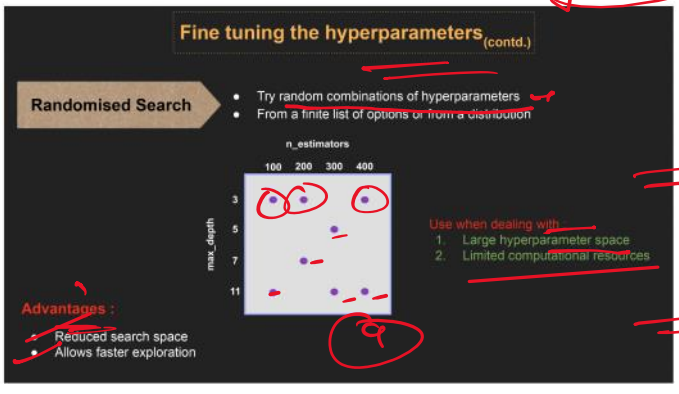
→ max_depth & n_estimators



→ Best params

Random Search CV → (16) → 9

Grid



Iter - 'D

Practice

Quiz time!

Time Left: 18s

If you're performing GridSearchCV on a random forest model for the parameters,

- **'max_depth' having 3 values and**
- **'min_samples' having 4 values**
- **for 10 cross-validations**

How many times model.fit() is called?

22 users have participated

	A	1440	0%
✓	B	120	100%
	C	40	0%
	D	30	0%

End Quiz Now

